Optimization Journey — From v1 prototype to production architecture

> *A short, candid record of the v1 design decisions, where each one stops being good enough, and the three upgrades that would replace them in a production rollout. Written to make the v1 → v2 evolution path explicit and reviewable.*

Why this document exists

The current build is a v1 prototype. v1 is not the final architecture — it's the deliberately scope-limited answer that hits the product's success criteria (signal selection, grounded explanation, no hallucination, top-3 failure modes) with the data and time available.

This doc covers three things:

1. **The v1 decisions** — what I picked and why each was the right call at this scope.
2. **The honest weaknesses** — where v1 stops being good enough at scale.
3. **The v2 upgrades** — three specific, named, industry-validated alternatives, with implementation sketch and trade-off analysis.

The goal is to make the evolution path mechanical rather than aspirational. v2 is data-bounded, not effort-bounded — the architecture is already shaped to absorb each upgrade without rewriting the rest of the system.

The v1 architecture, decision by decision

Selection: deterministic ranker

Z-score against a same-day-of-week 28-day rolling baseline, plus week-over-week and month-over-month deltas, weighted by metric business-importance, with a 3-level sort key (data-quality flags down, alerts up, score descending). Top-5 daily, top-8 weekly.

Explanation: grounded LLM

Claude API, system prompt cached, strict rules banning invented numbers and causal language, post-generation numeric validator that rejects ungrounded numbers and falls back to a deterministic template renderer.

Personalisation

A 1.3× multiplier on findings whose metric is in the customer's priority list. Two profiles (growth and scale) selected at signup.

Safety

Ratio-metric cap at ±500%, dedup table for Shopify total vs unattributed, data-quality flag on the last date of the dataset, three retries with exponential backoff on the LLM call.

Why these were the right v1 choices

The dataset gives 160 days of one anonymised tenant's data. Within that scope, every alternative I considered is either premature optimization or impossible:

- **STL decomposition needs at least two full cycles of the longest seasonality.** For weekly seasonality that's two weeks (fine). For annual seasonality that's two years (impossible with 160 days).
- **CausalImpact needs intervention markers and a valid control series.** The dataset has no labelled interventions and the channels in it overlap in customer base — no clean control.
- **Learned per-customer business weights need engagement data.** No clicks, no follow-up questions tracked, no "not relevant" signal. Zero training signal.
- **Cross-metric story grouping needs cardinality.** With ~200 series and 5–10 findings per day, grouping is useful but the gain is marginal at this scale.

The v1 approach is also the **most auditable** option. Every number that appears in a final report can be traced end-to-end through Python that runs the same way every time. Anything stochastic at the selection layer (LLM-as-judge, model-based scoring) would have made the system harder to defend against "why did the digest say this?" queries.

Where v1 stops being good enough

Six honest weaknesses. Each one is a real problem at production scale; each one is acceptable in a prototype.

W1 — The rolling baseline lags genuine level shifts

A new product line that doubles revenue permanently shows +large z for 28 days while the rolling mean catches up. That's a month of false alerts.

W2 — Z-score assumes approximate normality

Revenue and spend are log-normal in real ecommerce data. Z on raw values undercounts the rare big days because the standard deviation is inflated by them.

W3 — Seasonality is only weekly

Day-of-week works. Diwali, EOSS, Christmas, payroll-cycle 1st-and-15th spikes, fiscal year-end — none modelled. Every annual anomaly false-flags.

W4 — Mean and std are not robust

One catastrophic day in the baseline window inflates std for the next 27 days, suppressing real future signals. The statistical name for this is *masking*.

W5 — No causal inference

The hedged language is the defensive workaround for not having a counterfactual. The brand owner's question — "did the budget change drive revenue?" — gets a disclaimer, not an answer.

W6 — Findings are independent

Five correlated findings on the same day (Meta CPM up + CPC down + CTR down + ROAS down + spend down) get five bullets. They are one story.

---

v2 upgrades — three specific, named alternatives

Upgrade 1 — STL or Prophet for the baseline

Replaces: the rolling-mean-with-DoW-adjustment baseline in `_score_series_on_date()`.

What it is: decompose each series into `trend(t) + seasonality(t) + residual(t)` using STL (Seasonal-Trend decomposition using LOESS), or Prophet (Meta's open-source library that adds explicit holiday calendars and trend-change detection on top of STL). Anomaly detection runs on the **residual**, not the raw value.

Pipeline:

```
For each metric series, nightly:
  trend, seasonal, residual = STL(series, period=7).fit()
  baseline_for_today        = trend(today) + seasonal(today)
  z_score                   = (today_value - baseline_for_today) / std(residual)
  # everything downstream — business weights, sort, slice, profile rerank — stays identical
```

Why this is the right upgrade (addresses W1, W3, partly W2):

- Multiple seasonalities cleanly modelled (weekly via STL, annual + monthly + holidays via Prophet)
- Trend changes don't poison the anomaly signal — the trend component absorbs them
- The interface contract with the LLM doesn't change; the LLM still receives fact sentences with z-scores. Only the upstream computation is more honest.

Used in production by:

- **Netflix** — STL across the streaming-quality stack
- **LinkedIn** — Luminol open-source library uses a similar decomposition
- **Uber** — Prophet for capacity forecasting, custom anomaly logic on residuals
- **DataDog Watchdog** — proprietary STL-like decomposition with automated anomaly surfacing
- **Booking.com** — published their bootstrap-residual variant in their engineering blog

Trade-offs:

- STL needs ≥2 cycles of the longest seasonality. For annual seasonality that's 2 years of history — the current 160-day dataset can't fit annual STL. This is the literal reason it isn't in v1.
- ~100 ms per series per fit → ~20 seconds per nightly batch for 200 series. Acceptable.
- The residual is harder to explain than "today vs baseline." A UX pass is needed so brand owners read the z-score as "X std above what the model expected" rather than "X std above last month's average."

Upgrade 2 — CausalImpact for the attribution question

Replaces: nothing — it's an addition. The hedging language stays for passive observations; CausalImpact answers the active "did this change drive that outcome?" questions.

What it is: Bayesian Structural Time Series (Brodersen et al. 2015, Google Research). Given an intervention date and a control series, the model fits a state-space prediction of what would have happened without the intervention. The "lift" is the difference between observed and counterfactual, with credible intervals.

Pipeline:

```
For each flagged or detected intervention (budget change, creative refresh, channel launch):
  pre_window  = series before intervention_date
  control     = a series that wasn't affected by the intervention
  model       = CausalImpact(target=series, control=control, intervention=intervention_date).fit()
  posterior   = model.posterior_summary()
  # surface as a separate finding:
  #   "Meta retargeting budget +50% on Dec 4 — revenue +12% over counterfactual
  #    (95% CI: 7–17%, posterior P(lift>0) = 0.97)"
```

Why this is the right upgrade (addresses W5):

- Replaces the disclaimer with a decision-grade output.
- Posterior credible intervals + tail probabilities communicate uncertainty honestly without hiding the answer.
- Pairs naturally with the existing system — the digest can say "your Dec 4 budget change is the biggest contributor to this week's lift, estimated +12% (95% CI 7–17%)."

Used in production by:

- **Google** — introduced the technique; used internally for ad-product measurement
- **Walmart** — published a Walmart Global Tech blog walking through CausalImpact for incrementality
- **Lifesight, Measured, Recast** — all major incrementality / measurement vendors use BSTS variants
- **Meta** — uses related state-space models in Robyn (their open-source marketing-mix modelling library)

Trade-offs:

- Requires clean intervention markers, or a changepoint-detection step that finds them automatically (PELT, BOCPD)
- Requires a valid control series that wasn't affected by the intervention — non-trivial in adtech because brand effects spill across channels
- ~5 seconds per fit vs milliseconds for z-score
- Output is a posterior distribution, not a single number — needs careful UI so users interpret credible intervals correctly
- Doesn't help passive monitoring (v1's job) — it's a complement, not a replacement

Upgrade 3 — Cross-metric story grouping

Replaces: independent-finding presentation. Adds a clustering step between ranker and LLM.

What it is: before the top-N findings go to the LLM, cluster correlated movements into "stories." A cluster is a group of findings on the same date with the same source, similar magnitude, and a coherent direction signature.

Pipeline:

```
After top-N + profile rerank:
  for each pair of findings (i, j) on the same date:
      similarity[i,j] = (same_source × 0.4) + (same_channel × 0.3)
                      + (same_campaign × 0.2) + (matched_direction × 0.1)
  clusters = connected_components(similarity > 0.7)
  for each cluster of 2+ findings:
      story_finding = synthesize_story(cluster)   # tiny LLM call or rule-based template
  # the digest prompt receives stories at the top level, with member facts as supporting context
```

Why this is the right upgrade (addresses W6):

- Reduces cognitive load. "Auction pressure today — CPM +18%, CPC +14%, CTR -8%, ROAS -22%" is one bullet, not four.
- The most expensive cognitive work — connecting dots — moves from the reader to the system.
- Naturally amplifies the signal-to-noise win the top-N cap already provides.

Used in production by:

- **Anodot** — "stories" are the core product differentiator
- **Outlier.ai** (acquired by Salesforce) — built explicitly around correlated-anomaly grouping
- **DataDog Watchdog** — automated correlated-alert grouping in incident view
- **Splunk ITSI** — event correlation engine uses similar clustering

Trade-offs:

- Similarity metric needs care — same source + same campaign + matched direction is a fine first cut, but feature-level cosine similarity is more robust

- Risk of over-grouping: distinct stories that share a source get collapsed if the threshold is too loose. Per-cluster cardinality cap (e.g. max 4 metrics per story) mitigates this.
- Adds a small LLM step to synthesize the story headline. Modest prompt work, not architectural redesign. Falls back cleanly to a rule-based template ("auction pressure" / "creative fatigue" / "platform outage") when the LLM is unavailable.

Research path to these specific upgrades

Not speculative — each one comes from a specific reading thread.

For Upgrade 1 (STL / Prophet): the Booking.com engineering post on time-series anomaly detection (medium.com) which uses bootstrapped residuals on a similar baseline. That led to STL (Cleveland et al. 1990, still the canonical decomposition) and Prophet (Taylor & Letham 2017, Facebook's open-source extension that handles holidays natively). Netflix's "Anomaly Detection in the Streaming World" and LinkedIn's Luminol README confirm STL is the standard upgrade path from rolling z-score in production analytics platforms.

For Upgrade 2 (CausalImpact): the r=0.12 EDA finding was the catalyst — that's the number that forced the hedging rule, and "hedge it" isn't a complete answer. The reading thread: Brodersen et al. 2015 (the original Google Research paper) and Walmart's published case study on using BSTS for incrementality measurement. The conclusion: the "ad spend → revenue" attribution question is **already solved at the methodology level** — the gap in v1 is data (no intervention markers, no clean control), not method.

For Upgrade 3 (cross-metric story grouping): a typical day in the generated reports produces 3–5 findings from the same campaign with the same direction signature. That pattern is what Anodot's product literature calls a "story" and what DataDog Watchdog calls correlated alerts. Studying their product UX confirms this is a solved pattern, not invention.

Enterprise comparison — what production systems actually do

Company / Tool	Baseline method	Causation method	Grouping
Netflix	STL + LSTM forecasting	None public	Manual incident review
Booking.com	Bootstrapped z-score on residuals	A/B test platform	None public
Uber	Prophet + custom anomaly	Synthetic control + A/B	Tag-based
LinkedIn	Luminol (STL-based)	Causal inference toolkit	Correlation engine
DataDog Watchdog	Proprietary STL-like	None	Automated correlated alerts
Anodot	Multiple per-metric models	None	"Stories" — their core feature
Google	Mixed	CausalImpact (BSTS)	Internal
Walmart	Mixed	BSTS for incrementality	Internal
v1	Rolling z + DoW adjustment	Hedged language (r=0.12)	Dedup pairs only
v2 target	STL / Prophet	CausalImpact	Correlation clustering

The v2 target aligns with what serious production stacks do. It's not aspirational — it's the standard.

What stays the same in v2

The architectural skeleton is method-agnostic:

- **Python computes, LLM explains** — the central design choice is independent of the scoring method. Whether the baseline is rolling-mean or STL or Prophet, the LLM still receives pre-computed fact sentences.
- **Numeric grounding validator** — every number the LLM emits is still verified against the input. The scoring upstream doesn't affect this safety layer.
- **Retry + template fallback** — the resilience layer is independent of the model.
- **Prompt caching** — the system prompt doesn't change with the scoring method, so the caching win is preserved.
- **3-level sort key, top-N slicing, profile re-ranking** — all of these compose with whatever signal the upstream scorer produces.

The upgrades change **how the z-score is computed** and **what additional findings get surfaced**. They don't change the architecture.

Why v2 isn't built yet — four honest reasons

Ranked by honesty:

1. **The dataset doesn't support it.** STL needs 2+ cycles of the longest seasonality. Prophet wants 2+ years for holiday detection. CausalImpact needs labelled interventions and a clean control. v1's dataset has none of these.
2. **No engagement signal yet.** Half the personalisation roadmap depends on click data the source dataset doesn't include. Building learned weights against zero engagement data is fitting noise.
3. **Prototype scope.** The prototype-stage time budget buys v1 done well. Three weeks of clean implementation would buy v2 done well. v1 done well + v2 documented is more valuable than v2 half-built.
4. **v1 is the right scope for the current evaluation.** v1's job is to prove the analytical core works. v2's job is to prove the production rollout works. Different evidence required.

The grown-up framing: v1 is correct for the current scope; v2 is correct for the production rollout; both are real and the path from one to the other is mechanical.